

Data Engineer — Trainee

Take-home technical task

Time budget	3–4 hours. Spend no more than 4 hours; submit what you have.
Deadline	5 calendar days from the day you received this file.
Deliverable	One archive or one public repository (see section 6).
Level	Trainee. We do not expect production-grade code.

How we read your submission

We care much more about **how you reason** than about how much you finish. A partial solution with clear reasoning and honest notes about what you did not get to beats a complete solution you cannot explain. Every task below has a short “*explain*” part — those are not optional, and for several tasks they carry more weight than the code.

If a requirement is ambiguous, make a decision, write down the assumption you made, and move on. Ambiguity is deliberate in places. We want to see what you do with it.

Use of AI tools

You may use AI assistants. If you do, add a short section **AI_USAGE.md** to your submission saying which tool you used and for which parts. Using AI is not a problem; not being able to explain the result is. During the follow-up call we will ask you to walk through your own code line by line and to change it live.

1. The data

You are given a small mock dataset from a mobile-analytics vendor. Three CSV files are attached to the same email as this task. If you did not receive them, generate equivalent data yourself and say so.

File	Grain	Columns
events_raw.csv	one row per event delivery (duplicates expected)	event_id, user_id, app_id, event_name, event_time, ingested_at, country, media_source, campaign, revenue_usd, is_test
apps.csv	one row per app	app_id, app_name, platform, store_id, launched_on
campaign_costs.csv	one row per campaign per day	date, app_id, media_source, campaign, cost_usd, impressions, clicks

Known facts about the data, established by the team before you joined:

- The vendor may deliver the same event_id more than once. Repeated deliveries can differ in ingested_at and, occasionally, in revenue_usd — the vendor corrects revenue after the fact.
- event_time is UTC. ingested_at is UTC. They can differ by up to several days for late-arriving mobile events.
- country is sometimes an empty string, sometimes ' - ', sometimes lowercase.
- revenue_usd is a string in the CSV and may contain ' ', 'NULL' or a comma decimal separator.
- Rows with is_test = true come from internal QA devices and must never reach reporting.

2. SQL

Write standard SQL. Any dialect is fine (PostgreSQL, BigQuery, Snowflake, DuckDB) — state which one at the top of the file. Put all answers in `sql/`, one file per task.

Task 2.1 — De-duplication (required)

Produce a clean event table with exactly one row per `event_id`. When the same `event_id` arrives more than once, keep the **latest delivery** by `ingested_at`, so that a corrected revenue value wins over the original one. Exclude test traffic.

Explain: what happens to your query if two deliveries of the same event share the exact same `ingested_at`? Show how you make the result deterministic.

Task 2.2 — Window functions (required)

From the de-duplicated events, for every `app_id` and every calendar day, return:

- daily revenue,
- the running total of revenue since the app launched,
- the 7-day moving average of daily revenue,
- the day-over-day change in daily revenue, in percent.

Explain: days with no events at all are missing from your output. Does that matter for the 7-day moving average? If it does, describe (no code needed) how you would fix it.

Task 2.3 — Joining costs (required)

Build one daily table that puts revenue and cost side by side per `app_id`, `media_source` and `campaign`, and adds ROAS (revenue / cost).

Explain: there are campaigns with cost and no revenue, and campaigns with revenue and no cost. Which join did you choose and what does each of those two cases mean for the business? What did you do about division by zero?

Task 2.4 — Incremental load (required)

Write the statement that loads new and corrected events from a staging table into the clean table incrementally. It must be safe to re-run: running it twice over an overlapping period must not create duplicates and must not lose corrections.

Explain: which time column do you filter on to pick up new rows — `event_time` or `ingested_at` — and why? What is the smallest window you can reload without losing late events, and what does that choice cost you?

Task 2.5 — Data quality (optional, do it if you have time)

Write three checks you would run after every load, each as a query that returns zero rows when everything is fine. Say for each check whether you would fail the pipeline or just warn.

3. Python

Standard library and pandas (or polars) only. Put the code in python/.

Task 3.1 — Cleaning (required)

Write a function that reads `events_raw.csv` and returns a clean dataframe:

- `revenue_usd` as a float; unparseable and empty values become `0.0`, but count them and report the count,
- `country` normalised to an upper-case two-letter code; anything you cannot map becomes `'XX'`,
- timestamps parsed as timezone-aware UTC,
- test rows dropped,
- duplicates resolved with the same rule as in task 2.1.

The function must not crash on a malformed row. Bad rows are quarantined, not silently dropped: return them separately.

Task 3.2 — A small pipeline (required)

Wrap 3.1 into a script that can be run from the command line:

```
python -m pipeline.run --input data/ --output out/ --since 2026-01-01
```

It should:

- write the clean events as Parquet, partitioned by event date,
- write the quarantined rows separately,
- print a one-line summary: rows in, rows out, duplicates removed, rows quarantined,
- be re-runnable over the same period without corrupting previous output.

Explain: in three or four sentences, what your script does if it dies half-way through writing the output, and what you would change to make that safe.

Task 3.3 — Reading someone else's code (required)

A colleague wrote this. It works on the sample file and fails in production.

```
import pandas as pd

def load_daily(path, day):
    df = pd.read_csv(path)
    df = df[df.event_time.str.startswith(day)]
    df["revenue_usd"] = df["revenue_usd"].astype(float)
    totals = {}
    for i, row in df.iterrows():
        key = row["app_id"] + "-" + row["media_source"]
        totals[key] = totals.get(key, 0) + row["revenue_usd"]
    return pd.DataFrame(
        [{"key": k, "revenue": v} for k, v in totals.items()]
    ).sort_values("revenue", ascending=False)
```

List the problems you see — correctness, performance, robustness — ranked by how much they matter. Rewrite the function. You do not need to keep the same signature; if you change it, say why.

4. Modelling and design

Short written answers, in `ANSWERS.md`. Four to eight sentences each — we are not looking for essays, and copied textbook definitions score zero.

4.1 Star schema

Sketch a star schema for this data: name the fact table, its grain, its measures, and the dimensions. State the grain in one sentence, explicitly.

4.2 A dimension that changes

A campaign is renamed mid-flight. Yesterday's report was built under the old name. Describe what you would do so that historical numbers stay correct while new rows use the new name, and say what that costs you in query complexity.

4.3 Idempotency

Explain idempotency to a marketing manager who does not write code, in three sentences. Then explain to an engineer why an `INSERT ... SELECT` into a daily table is not idempotent and what the usual fixes are.

4.4 Late data

An event happened on Monday and arrives on Thursday. Monday's report has already been sent to the client. Walk through what your pipeline does and what you tell the client.

4.5 Partitioning

You are asked to partition the fact table. Which column do you partition on, and why that one rather than the obvious alternative? Name one query pattern your choice makes slower.

5. Debugging scenario

Monday morning. The daily revenue dashboard shows **0** for Sunday for one app out of twelve. The pipeline reported success. Nobody deployed anything over the weekend.

Write, in order, the first five things you check — and for each one, say what a *yes* and a *no* would tell you. We are reading this as a test of how you narrow down a problem, not whether you guess the right cause.

6. What to submit

- `sql/` — one file per SQL task, each with a comment header naming the dialect.
- `python/` — your code, runnable.
- `ANSWERS.md` — the written parts, including every “*explain*” above.
- `ASSUMPTIONS.md` — every assumption you made, and anything you did not finish.
- `AI_USAGE.md` — if you used an AI assistant.

A repository link is fine. Do not include the input CSVs in the submission.

How we score

Correctness of SQL	25%
Python: correctness and readability	20%
Quality of the written explanations	30%
Handling of edge cases, late data and re-runs	15%
Clarity of the submission itself	10%

The written explanations are weighted highest on purpose. Good luck.